



Document de référence · v1.0

Prompt Library MOTO Editor

La bibliothèque de prompts officielle du projet : des modèles prêts à l'emploi pour travailler avec les trois IA de l'équipe (Claude Code, Qwen, Copilot), les moteurs internes (XENO-SSS[∞], Ollama) — et des prompts universels réutilisables pour tout projet. Chaque prompt est testé, contextualisé et honnête sur ses limites.

Septembre 2026	IA couvertes	Prompts	Doctrine
Édition v1.0	Claude Code · Qwen · Copilot · Ollama	40+ modèles prêts à copier	« Dire vrai, dire mesuré, dire utile »
Prompt Library · Septembre 2026			1 / 13

Sommaire

1. Les 6 piliers d'un bon prompt + gabarit universel	p.3
2. Méta-prompts & techniques de relance	p.4
3. Prompts généraux — Code & débogage	p.5
4. Prompts généraux — Architecture, conception & revue	p.6
5. Prompts généraux — Documentation, tests & apprentissage	p.7
6. Prompts généraux — Contenu, productivité & résolution	p.8
7. Prompts MOTO Editor — Claude Code (développement)	p.9
8. Prompts MOTO Editor — Qwen (architecture & documents)	p.10
9. Prompts MOTO Editor — moteurs IA internes (XENO / Ollama)	p.11
10. Anti-prompts & dépannage	p.12
11. Checklist, glossaire & conclusion	p.13

Mode d'emploi de cette bibliothèque

Légende des badges

- **UNIVERSEL** — réutilisable pour tout projet.
- **CLAUDE CODE** — développement sur le dépôt.
- **QWEN** — architecture, documents, conseil.
- **XENO/OLLAMA** — moteurs IA internes.

Comment utiliser un prompt

- Copier le bloc tel quel.
- Remplacer les crochets **[comme ceci]** par vos valeurs.
- Ajouter le contexte manquant si nécessaire.
- Vérifier la sortie avant de l'appliquer — toujours.

Règle d'or du projet

- Aucune IA n'applique seule : l'utilisateur valide.
- Jamais de secret (PAT, clé, mot de passe) dans un prompt.
- Un prompt = une tâche claire. Pas plus.

PRIORITÉ ABSOLUE

« Un bon prompt vaut mieux que dix relances. »

Dix secondes de structuration avant l'envoi font gagner dix minutes de corrections après. Chaque modèle de cette bibliothèque suit le même squelette : contexte → rôle → tâche → contraintes → format → validation.

1. Les 6 piliers d'un bon prompt

Peu importe l'IA (Claude, Qwen, Copilot, Ollama...), un prompt efficace contient systématiquement ces six éléments. En manquer un seul est la première cause de réponse hors sujet.

1 Contexte

Qui suis-je, quel projet, quelle stack, quelle version. L'IA ne devine rien.

2 Rôle

« Tu es un ingénieur senior MAUI... » : cadre le niveau et le ton de la réponse.

3 Tâche

Un seul verbe d'action clair : corrige, propose, explique, compare, génère.

4 Contraintes

Ce qu'il ne faut PAS faire : fichiers interdits, pas de dépendance, style imposé.

5 Format

Code, tableau, liste, 3 options max, markdown, PDF... la forme avant le fond.

6 Validation

« Si une info manque, pose la question avant d'agir. » — évite les suppositions.

Le gabarit universel

UNIVERSEL — À garder sous le coude pour tout projet, toute IA.

```
[CONTEXTE] Projet : [nom]. Stack : [technologies + versions]. État actuel : [résumé en 2 lignes].
[RÔLE] Tu es un expert senior en [domaine], rigoureux et honnête.
[TÂCHE] [Un verbe d'action + un objectif précis et mesurable.]
[CONTRAINTES] Ne pas toucher à [X]. Respecter [convention de code]. Pas de dépendance nouvelle.
[FORMAT] Réponds sous forme de [code / liste / tableau / 3 options avec trade-offs].
[VALIDATION] Si une information manque ou si une hypothèse est nécessaire, pose la question avant d'agir. Ne suppose pas.
```

Le principe de progressivité

Petite tâche → prompt direct

- « Corrige cette erreur CS8618 dans ce fichier, changement minimal, explique en 2 lignes. »
- Pas besoin de contexte étendu pour un fix local.

Grande tâche → découper d'abord

- « Avant de coder : propose un plan en 3-5 étapes. Attends ma validation avant chaque étape. »
- Doctrine MOTO : jamais d'auto-application, chaque étape est validée par l'humain.

💡 Pour les tâches longues, demander d'abord le plan, puis faire valider chaque étape : c'est exactement la méthode de travail utilisée sur MOTO Editor entre le fondateur et Claude Code.

2. Méta-prompts & techniques de relance

Les méta-prompts sont des prompts qui améliorent... d'autres prompts. Ce sont les plus rentables de toute la bibliothèque.

UNIVERSEL L'améliorateur de prompt

Voici mon prompt : « [coller votre brouillon] ».
Améliore-le pour qu'il produise la meilleure réponse possible : ajoute le contexte manquant, précise le format attendu, signale les ambiguïtés. Renvoie-moi la version améliorée et la liste des questions que tu as dû trancher.

UNIVERSEL Le mode socratique (l'IA vous interroge)

Avant de répondre, pose-moi toutes les questions nécessaires pour être certain de bien comprendre ma demande (maximum 5 questions, les plus importantes d'abord). Attends mes réponses avant de produire quoi que ce soit.

UNIVERSEL La relance cadrée (quand l'IA dévie)

Stop. Tu t'éloignes de la demande initiale. Rappel du cadre :
- Tâche : [reformuler en une phrase]
- Ce qui est déjà validé : [lister]
- Ce que je veux maintenant : [une seule chose précise]
Reprends uniquement à partir de là, sans recommencer le reste.

UNIVERSEL L'auto-vérification avant livraison

Avant de me donner ta réponse finale : relis-toi comme si tu étais un relecteur critique. Liste les erreurs possibles, les cas limites oubliés et les hypothèses que tu as faites sans me le dire. Corrige ta réponse en conséquence, puis présente la version corrigée avec un court paragraphe « ce que j'ai corrigé en me relisant ».

UNIVERSEL Le test d'honnêteté

Si tu n'es pas sûr de ta réponse, dis-le explicitement et indique ton niveau de confiance (faible / moyen / fort) avec la raison. Je préfère une honnête incertitude à une réponse inventée.

💡 La revue croisée entre IA fonctionne avec le même prompt d'auto-vérification : donner la proposition de Claude Code à Qwen avec « critique cette approche, trouve les failles », et inversement. C'est cette méthode qui a intercepté plusieurs bugs sur les docks de MOTO Editor.



3. Prompts généraux — Code & débogage

UNIVERSEL Diagnostic avant correction

Voici l'erreur exacte : [message complet]
Voici la stack trace : [coller]
Voici le code concerné : [coller]
Donne-moi les 3 causes les plus probables, classées de la plus à la moins probable. Pour chacune : l'explication, et une vérification simple pour confirmer ou infirmer. Ne corrige rien pour l'instant.

UNIVERSEL Correction minimale

Corrige ce bug avec le changement le plus petit possible. Explique : 1) la cause racine en 2 lignes, 2) ce que tu changes exactement, 3) pourquoi c'est le fix minimal. Aucun refactoring, aucune amélioration annexe, même tentante.

UNIVERSEL Explication pour débutant

Explique ce code comme à un débutant complet : d'abord une analogie du quotidien, puis ligne par ligne ce que fait chaque partie, enfin les 2 pièges classiques que rencontrent les débutants avec ce genre de code. Pas de jargon non défini.

UNIVERSEL Reproduction fiable d'un bug

Ce bug apparaît de façon aléatoire : [description]. Propose : 1) une liste de vérifications pour isoler les conditions d'apparition, 2) un scénario de reproduction pas à pas, 3) les logs ou traces à capturer pendant le test. L'objectif est un bug reproductible à 100 %, pas encore un fix.

UNIVERSEL Migration de version sans casse

Je dois migrer [projet] de [ancienne version] vers [nouvelle version]. Établis : 1) les breaking changes qui me concernent réellement (au vu de ce code : [coller]), 2) l'ordre des étapes de migration, 3) le test à faire après chaque étape pour détecter une régression immédiatement.

💡 Enchaînement gagnant : **diagnostic** → **reproduction** → **correction minimale**, dans cet ordre. Demander directement un fix sans diagnostic est la première cause de « fix qui déplace le bug ».

4. Prompts généraux — Architecture, conception & revue

UNIVERSEL Trois options avec trade-offs

Pour résoudre [problème], propose 3 approches architecturales différentes. Pour chacune : le principe en 3 lignes, les avantages, les inconvénients, le coût de mise en œuvre (faible/moyen/fort), et le cas où elle devient le mauvais choix. Termine par ta recommandation argumentée pour MON contexte : [contexte : équipe solo, budget mince, etc.].

UNIVERSEL Décision technique documentée (ADR)

Rédige une décision d'architecture (ADR) au format standard : Titre, Statut, Contexte, Décision, Alternatives rejetées et pourquoi, Conséquences (positives, négatives, risques). Décision à documenter : [décrire]. Sois concis : ce document sera relu dans 6 mois par quelqu'un qui n'était pas là.

UNIVERSEL Revue de code exigeante

Tu es un relecteur senior strict mais constructif. Relis ce code et produis : 1) les bugs réels (pas les préférences de style), 2) les problèmes de sécurité, 3) les problèmes de performance sur les chemins critiques, 4) les 3 améliorations les plus rentables. Classe chaque point par gravité : bloquant / important / suggestion. Ne propose rien d'autre.

UNIVERSEL Plan de refactoring incrémental

Je dois refactoriser [module] sans jamais casser l'application en production. Propose un plan en petites étapes indépendantes, chacune : 1) réversible, 2) vérifiable par un test, 3) commitable séparément. Commence par l'étape qui réduit le plus de dette avec le moins de risque.

UNIVERSEL Détection de dette technique

Analyse ce code et liste la dette technique réelle (pas théorique) : pour chaque point, où, pourquoi c'est un problème MAINTENANT, et le coût estimé de correction (heures). Classe par ratio impact/coût. Je veux le top 5, pas une liste exhaustive.

💡 Le prompt « 3 options avec trade-offs » est celui qui a servi à choisir l'approche de la barre de titre de MOTO Editor après 3 échecs : arrêter de forcer une piste, poser les alternatives sur la table, et trancher sur des faits.

5. Prompts généraux — Documentation, tests & apprentissage

UNIVERSEL README auto-généré

Rédige un README.md pour ce projet [coller structure + fichiers clés] : description en 1 phrase, installation, usage rapide, architecture en 5 lignes, licence. Ton direct, pas de superlatifs, pas de « révolutionnaire ».

UNIVERSEL Changelog Added/Changed/Fixed

À partir de cette liste de commits : [coller], rédige un changelog au format Added / Changed / Fixed. Regroupe par impact utilisateur, pas par commit. Phrases courtes, sans jargon interne.

UNIVERSEL Génération de tests unitaires

Écris les tests unitaires de cette classe : [coller]. Couvre : cas nominal, cas limites (vide, null, maximum), et un cas d'erreur attendu. Un test = un comportement, nom explicite. Framework : [JUnit/PHPUnit/pytest..].

UNIVERSEL Le 80/20 d'une technologie

Je dois être opérationnel en [technologie] pour [objectif précis]. Donne-moi : les 20 % de concepts qui couvrent 80 % des cas d'usage, l'ordre d'apprentissage optimal, les 3 ressources de référence, et les pièges qui font perdre du temps aux débutants.

UNIVERSEL L'analogie éclair (comprendre un concept)

Explique-moi [concept technique] avec une analogie du quotidien, sans jargon. Puis montre où l'analogie s'arrête (ses limites), car je veux comprendre le vrai mécanisme, pas une image approximative.

UNIVERSEL Documentation inline propre

Ajoute une documentation inline (XML docs / docstrings) à ce code : [coller]. Règles : décrire l'intention, pas paraphraser le code ; documenter les paramètres non évidents et les valeurs de retour ; signaler les effets de bord. Pas de commentaire sur les lignes évidentes.

💡 Le prompt 80/20 est celui qu'un fondateur non-développeur devrait utiliser en premier sur chaque nouvelle technologie du projet — avant de demander du code. Comprendre la carte avant de conduire.

6. Prompts généraux — Contenu, productivité & résolution

UNIVERSEL Résumé honnête

Résume ce texte en [X] lignes : l'idée principale, puis les 3 points clés, puis ce que le texte NE DIT PAS mais laisse entendre. Aucun ajout d'information extérieure.

UNIVERSEL Réécriture de ton

Récris ce texte avec le ton suivant : [direct / chaleureux / formel / pédagogique]. Garde exactement le même contenu factuel. Signale toute information que tu as dû sacrifier pour tenir le format.

UNIVERSEL Les premiers principes

Décompose ce problème en ses composants fondamentaux, sans reprendre les solutions habituelles : [problème]. Puis reconstruis une solution à partir de zéro. Je veux comprendre pourquoi on fait « comme ça d'habitude », et si c'est justifié.

UNIVERSEL Plan d'action 72 h

Je dois accomplir [objectif] en 72 heures avec [ressources]. Propose un plan heure par heure pour les 8 premières heures, puis par demi-journées. Identifie le point de blocage le plus probable et comment le prévenir.

UNIVERSEL La matrice de décision

Je dois choisir entre [option A] et [option B] pour [contexte]. Construis un tableau comparatif avec les critères qui comptent réellement dans ma situation : [coût, temps, risque, réversibilité...]. Attribue une note honnête par critère (pas de notes moyennes artificiellement équilibrées) et conclus clairement.

UNIVERSEL L'avocat du diable

Voici ma décision / mon plan : [décrire]. Attaque-le : trouve les 3 raisons pour lesquelles il pourrait échouer, les hypothèses fragiles, et ce que je refuse de voir. Sois dur mais factuel. Termine par : « malgré tout, ça tient si... » — ou dis-moi honnêtement si ça ne tient pas.

💡 « L'avocat du diable » est le prompt le plus utile pour un fondateur solo : personne d'autre ne challenge vos décisions. L'IA le fait sans ego — à condition de le demander explicitement et de ne pas la remercier trop vite.



7. Prompts MOTO Editor — Claude Code (développement)

Claude Code travaille sur le dépôt [Tomoushie/MOTO-Editor](#) (.NET MAUI / WinUI 3). Ces prompts intègrent les règles permanentes du projet : additif, compilable, vérifié, sans secret.

CLAUDE CODE Vérification de build systématique

Avant toute modification, lance : `dotnet build Moto.Editor/Moto.Editor.csproj`
1) Si erreurs : corrige-les d'abord, une par une, changement minimal.
2) Si 0 erreur : liste le nombre de warnings par projet et compare avec l'objectif (< 50).
3) Committe uniquement si le build passe. Ne jamais commiter un build cassé.

CLAUDE CODE Méthode complète de correction de bug

Bug signalé : [description précise + étapes de reproduction].
Méthode imposée, dans cet ordre :
1. Reproduire ou identifier le scénario exact.
2. Diagnostic : cause racine, pas symptôme. Citer le fichier et la ligne.
3. Correctif minimal, additif si possible, sans refactoring.
4. Build de vérification.
5. Résumé : ce qui était cassé, pourquoi, ce qui change.
Ne passe à l'étape suivante que si la précédente est confirmée.

CLAUDE CODE Revue croisée avant commit

Avant de committer, relis tes propres modifications comme un relecteur externe :
- Cas limites : fenêtre redimensionnée très petite, valeurs nulles, dossiers vides.
- Doubles déclenchements d'événements (abonnements multiples).
- Fuites : événements non désabonnés, ressources non libérées.
Liste les problèmes trouvés et corrige-les AVANT le commit.

CLAUDE CODE Respect de l'architecture de délégation

Rappel d'architecture strict avant cette tâche :
- MOTO Editor : édite, affiche, ouvre, sauvegarde, lance des commandes.
- MOTO AI : assiste, prédit, propose, route – jamais d'auto-application.
- XENO-SSS[∞] : opérations structurées projet complet.
La tâche : [décrire]. Vérifie que ta proposition reste dans la bonne couche. Si la tâche traverse deux couches, propose le découpage avant de coder.

CLAUDE CODE Ajout d'un réglage data-driven

Ajoute le paramètre [nom] au SettingsCatalog : type [bool/int/enum/string], défaut [valeur], catégorie [parmi les 15].
Règles : déclaration data-driven dans le catalogue, persistance automatique dans settings.json, application live via SettingsApplier, aucun code UI ad hoc. Style XML docs, `_camelCase`, < 40 lignes par fichier touché si possible.

💡 La « revue croisée avant commit » est le prompt qui a intercepté le bug des docks superposés (bouton changer de côté) : sans lui, il passait en production.

8. Prompts MOTO Editor — Qwen (architecture & documents)

Qwen tient le rôle d'architecte, de rédacteur de documents et de relecteur indépendant. Il ne touche pas au dépôt : sa valeur est le regard extérieur.

QWEN Mise à jour du journal de bord

Voici l'historique de la dernière session de développement : [coller l'export].
Mets à jour le journal de bord : 1) ce qui a été livré (avec état /), 2) les décisions prises et pourquoi, 3) les bugs connus restants, 4) les leçons apprises à enregistrer comme règles permanentes, 5) les 3 prochains chantiers naturels. Format : synthétique, tableaux, sans reformulation inutile.

QWEN Revue critique d'une proposition de Claude Code

Voici la proposition d'implémentation de Claude Code pour [fonctionnalité] : [coller].
Contexte projet : local-first, MAUI/WinUI 3, pas de dépendance lourde, doctrine « façade simple, architecture complexe ».
Critique cette approche : failles, cas limites, dette future cachée, incohérence avec l'architecture existante. Termine par : approuver / approuver avec modifications / rejeter avec alternative.

QWEN Génération d'un document PDF conforme à la charte

Génère un document PDF (HTML prêt à imprimer) pour MOTO Editor : [type de document].
Charte graphique imposée : fond blanc, accent #007ACC, accent-dark #005A9E, police Inter (fontsource), logo hexagonal « M », pages A4 avec pied de page et numéro de page, toolbar d'impression.
Ton : « dire vrai, dire mesuré, dire utile » – honnête sur les limites, pas de superlatifs. Structure : couverture, sommaire, sections, tableaux, conclusion.

QWEN Arbitrage entre deux options techniques

Claude Code propose l'option A : [résumé]. Une autre piste serait l'option B : [résumé].
Contexte : équipe = 1 fondateur non-développeur + 3 IA, budget mince, objectif v1.0 publique.
Tranche : tableau comparatif (effort, risque, dette, réversibilité), recommandation claire, et « ce que je sacrifie en choisissant cette option ». Ne pas recommander une option « parce qu'elle est moderne » : la plus simple qui tient la route gagne.

QWEN Synthèse de documents concurrents

Voici [N] documents sur le même sujet produits par différentes IA : [coller / décrire].
Produis une synthèse unique : points d'accord (faits), points de divergence (avec qui dit quoi), et ta position argumentée. Aucune information inventée : si les sources se contredisent, le dire explicitement.

💡 Le prompt « charte graphique » est volontairement exhaustif : c'est lui qui garantit que chaque nouveau document (SWOT, budget, politique de confidentialité...) ressemble exactement aux précédents.

9. Prompts MOTO Editor — moteurs IA internes (XENO / Ollama)

Ces prompts s'adressent aux IA qui tournent DANS l'éditeur : Ollama (modèles locaux) et XENO-SSS[®] (pipeline Scanner → Analyser → Synthétiser → Connector → Validator). Leur particularité : des instructions qui rappellent la délégation et la non-auto-application.

XENO/OLLAMA Génération avec validation (jamais d'auto-apply)

Tâche : [décrire la génération ou modification].
Règles absolues :
1. Produis une proposition de modification (diff), pas un fichier appliqué.
2. Liste les fichiers impactés et les dépendances ajoutées/modifiées.
3. Signale toute incertitude : si le contexte du projet est insuffisant, demande les fichiers manquants.
4. Termine par les étapes de validation que l'utilisateur devra faire (build, test visuel).

XENO/OLLAMA Analyse de projet structurée

Analyse ce projet selon le pipeline standard :
- Scanner : liste fichiers, projets, dépendances réelles (pas supposées).
- Analyser : architecture détectée, patterns reconnus, points de fragilité.
- Sortie attendue : rapport JSON structuré {fichiers, dépendances, patterns, risques[]}, chaque risque avec sévérité et justification.
Interdit : proposer des corrections dans ce rapport. Analyse uniquement.

XENO/OLLAMA Complétion locale sobre (modèles modestes)

Complète la suite de ce code. Contraintes : même style que l'existant, mêmes conventions de nommage, pas de dépendance nouvelle, pas d'import non utilisé. Si la suite logique dépasse ce que le contexte permet de deviner avec certitude, complète seulement la ligne en cours et arrête-toi.

XENO/OLLAMA Revue de cohérence (ConsistencyScanner)

Vérifie la cohérence du projet : 1) fichiers référencés mais absents (« fantômes »), 2) projets déclarés sans fichiers de code, 3) dépendances NuGet déclarées mais jamais utilisées, 4) namespaces incohérents avec l'arborescence. Sortie : liste par catégorie avec chemin exact et action suggérée (supprimer / ajouter / ignorer). Aucune action automatique.

⚠ Règle sécurité (rappelée dans chaque prompt interne)

Les modèles locaux voient ce que vous leur donnez. Ne collez jamais de secrets dans une requête IA, même locale. Si cela arrive : révoquez le secret, puis réinitialisez la mémoire Cortex (Settings → IA → Réinitialiser).

10. Anti-prompts & dépannage

10.1 Ce qu'il ne faut PAS demander

✗ « Corrige tout le code »

Aucune IA ne peut « tout corriger » : périmètre infini = résultat imprévisible. Une tâche, un périmètre, un critère de succès.

✗ « Fais-le directement dans le projet »

Doctrine MOTO : jamais d'auto-application. Toujours une proposition, toujours une validation humaine. C'est une règle d'architecture, pas de politesse.

✗ « Invente une solution élégante »

« Élégant » est subjectif. Demandez : simple, minimal, testable, réversible. Des critères vérifiables.

✗ Coller un secret « juste pour voir »

PAT, clé API, mot de passe : même en local, c'est une trace dans des logs, une mémoire, un historique. Jamais.

10.2 Dépannage : quand l'IA dérape

Symptôme	Prompt de secours
L'IA s'emballe (réponse trop longue, hors sujet)	« Stop. Réponds en 5 lignes maximum : 1) ce que tu as compris de ma demande, 2) ta réponse, 3) ce qui t'a fait douter. »
Elle invente des faits (hallucination)	« Sépare strictement : ce qui vient du contexte que je t'ai donné, et ce que tu supposes ou inventes. Marque chaque affirmation [VÉRIFIÉ] ou [SUPPOSÉ]. »
Elle tourne en rond sur un bug	« Nous tournons en rond. Récapitule : hypothèses testées, hypothèses écartées, et pourquoi. Puis propose UNE hypothèse nouvelle que nous n'avons pas encore testée. »
Deux IA se contredisent	À une troisième IA : « Voici deux réponses opposées au même problème [A et B]. Analyse laquelle est correcte et pourquoi, ou si les deux sont valables dans des contextes différents. »
Elle refuse une tâche légitime	« Reformule ce que tu comprends de ma demande et la raison exacte de ton refus. Si c'est un malentendu sur l'intention, voici la reformulation : [...]. »

💡 Le symptôme le plus fréquent sur MOTO Editor : l'IA qui force une piste déjà morte (la barre bleue, 3 tentatives). Le bon réflexe n'est pas un meilleur prompt de fix — c'est le prompt « 3 options avec trade-offs » qui force la remise à plat.

11. Checklist, glossaire & conclusion

Checklist avant d'envoyer un prompt (30 secondes)

✓	Vérification
☐	Le contexte est présent : projet, stack, version, état actuel.
☐	Une seule tâche claire, avec un verbe d'action.
☐	Les contraintes explicites : ce qu'il ne faut PAS faire.
☐	Le format de réponse attendu est précisé.
☐	La clause de validation : « pose la question si une info manque ».
☐	Aucun secret, aucune donnée sensible dans le prompt.
☐	Je sais comment je vérifierai la réponse avant de l'appliquer.

Glossaire minimal

Termes techniques ● Prompt : instruction envoyée à une IA. ● Contexte : fenêtre d'information que l'IA peut traiter d'un coup. ● Token : unité de texte (~¾ de mot) ; la monnaie du contexte. ● Température : réglage de créativité/aléatoire de la réponse. ● System prompt : instructions de fond qui cadrent toute la conversation.	Concepts de méthode ● Hallucination : réponse fausse présentée avec assurance. D'où la vérification systématique. ● Few-shot : donner des exemples dans le prompt avant la demande. ● Revue croisée : faire critiquer la sortie d'une IA par une autre. ● Journal de bord : mémoire persistante du projet, mise à jour à chaque session. ● Anti-prompt : demande à ne jamais faire (voir §10).
--	--

CONCLUSION — UNE PHRASE

« Le prompt est un contrat : plus il est précis, plus la réponse est fiable. »

Cette bibliothèque est vivante : chaque session de développement qui révèle un bon prompt (ou un mauvais) doit l'enrichir — au format Added / Changed / Fixed, comme tout le reste du projet. Prochaine révision prévue : après l'écran Réglages, pour ajouter les prompts de test des 300 paramètres.